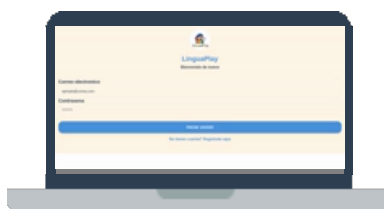




## Web performance and quality report

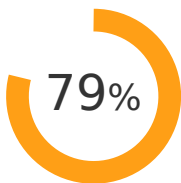


<https://linguplay-test.onrender.com/>

## Table of contents

|                         |    |
|-------------------------|----|
| Summary                 | 3  |
| Tips and best practices | 4  |
| Accessibility           | 4  |
| Browser rendering       | 5  |
| Cache policy            | 6  |
| Data amount             | 7  |
| SEO                     | 8  |
| Security                | 10 |

# Summary



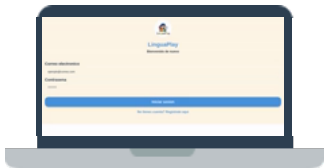
Issues



Improvements



Successes



SIMULATED VISITOR:

Desktop



USA - West

10.0/2.0Mbps (Latency: 28 ms)

Largest Contentful Paint



Total Blocking Time



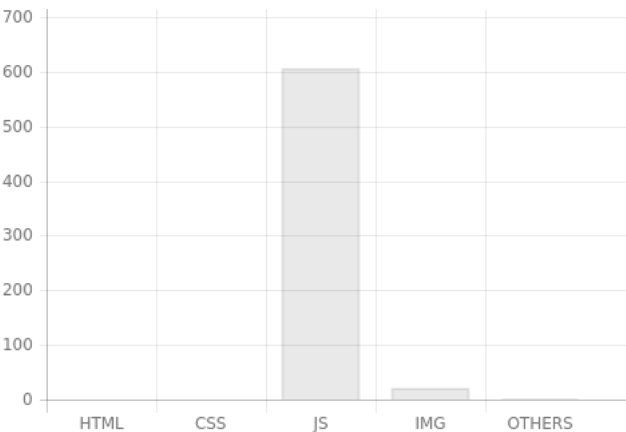
Cumulative Layout Shift



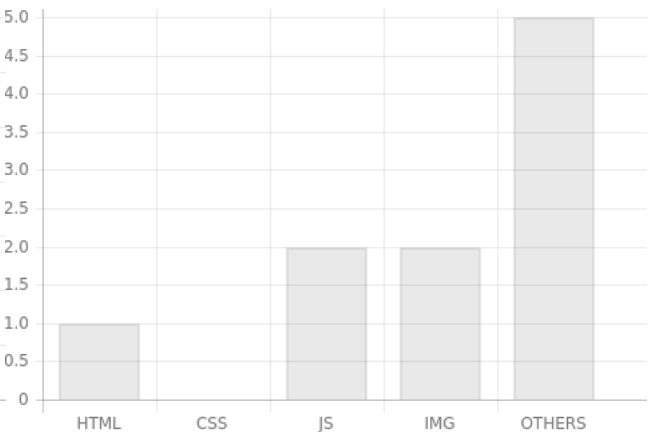
Speed Index



## Weight by resources type



## Requests by resources type



## Detected technologies



CloudFlare

# Tips and best practices

## Accessibility

### Your Optimization Priorities

0/100

#2381

#### ! Explain the purpose of each form field

Clarify the purpose of each field will facilitate the user experience on your website.

A form is composed of several fields that must be the most explicit possible for the user to quickly understand their function.

##### Define a label

You should prefer using the `label` tag:

```
<label for="name">Fill your name:</label>
<input id="name" type="text" name="name">
```

Example

Otherwise, you can use the `aria-label` or the `title` (not supported by all screen readers) attributes. [Read more](#). Note that [using the placeholder attribute is insufficient](#).

This page contains **2 fields without any explanations** :

- `<input placeholder="ejemplo@correo.com" autocapitalize="none" autocomplete="on" autocorrect="on" dir="auto" rows="1" spellcheck="true" virtualkeyboardpolicy="auto" class="css-11aywtz r-6taxm2 r-1xfd6z..."`
- `<input placeholder="*****" autocapitalize="sentences" autocomplete="on" autocorrect="on" dir="auto" rows="1" spellcheck="true" virtualkeyboardpolicy="auto" class="css-11aywtz r-6taxm2 r-1xfd6ze r-r..."`

### Your Optimization Priorities

**0/100**

#2353

#### **Defer parsing of JavaScript**

JavaScript can [significantly slow down](#) a page display, especially if it is necessary to download an external script.

Defer the use of JavaScript as much as possible to provide a faster start for the page display.

##### **How can I fix this?**

First of all, distinguish what portions of your JS is critical and must be loaded as soon as possible, and put them in a specific external file. Keep this file as streamlined as possible, and defer the parsing or execution of all other JS files.

Use one of the methods below to defer parsing for external JavaScript files:

- use the [async](#) attribute;
- use the [defer](#) attribute;
- append the script to the DOM in JavaScript during the onload event;
- make sure your scripts are placed at the bottom of the page (ideally at the end of the body).

525.8KiB of JavaScript is parsed during initial page load. Defer parsing JavaScript to reduce blocking of page rendering.

- [t.contentsquare.ne\[...\].js](#) (525.8KiB)

## Cache policy

---

### Your Optimization Priorities

**60/100**

#2437

#### Set a far future cache policy in 2 requests

Defining several days of cache retention for your static resources will reduce the load on your server.

##### The Expires header explained

Some of your resources use the **Expires** HTTP header to get an effective caching policy—this is a best practice. However, you should consider improving its configuration to make the most of the caching mechanisms. Here is an example of the **Expires** HTTP header:

```
Expires: Thu, 25 Dec 2014 20:00:00 GMT
```

Example

When you **deploy a new version of your website, remember to rename static resources** that have been modified. If you do not change their names, your users will keep resources corresponding to the old versions stored in their caches, and they may find themselves on an unstable version of your page. For example:

```
myresource.min.20140101.js
```

Example

Read [the Yahoo! guidelines](#) on this subject.

##### Recommended Expires header setting

We recommend setting the **Expires** HTTP header, so the date is between 2 days and 1 year.

This page contains 2 resources that **do not have a far expiration date**:

- [linguplay-test.onrender.com/\\_expo/stati\[...\]62ef4ded4f9b268.js](#)
- [linguplay-test.onrender.com/assets/asse\[...\]9cadfe631682fe.jpg](#)

### Your Optimization Priorities

**0/100**

#2436

**! 1 image is resized on browser side**

If your images are larger than their display area, the browser will download unnecessary data (and perform unsupervised resizing).

**Avoid resizing images on the browser side**

Resizing images on the browser side to reduce their rendering size is not recommended.

When the browser needs to display an image on your page, it does everything it can to adapt it to its rendering surface. If the image is too large, it will reduce it.

Provide images adapted to the display dimensions to prevent unnecessary data from being sent over the network, which reduces page loading time.

And because embedded browser algorithms are not as good as those of image manipulation tools, you will get a more satisfying visual result by resizing your images upfront, rather than letting the browser do it.

**Serve Responsive Images**

Several methods exist, to serve images adapted to the browser regardless of screen resolution or device pixel density. We recommend reading the following resources:

- ["Responsive images" on the Mozilla Developer Network](#)
- [Picturefill, to start using the <picture> element in browsers that do not support it](#)
- [RICG, group of developers working on responsive images](#)

Don't resize the following image:

- [linguplay-test.onrender.com\[...\]/1682fe.jpg](#) (displayed size: 100x100)

## Your Optimization Priorities

0/100

#78

### ! Define at least <h1> and <h2> tags in your page

We recommend putting page keywords in at least the h1 and h2 tags. Search engines use the h1, h2, and h3 tags for SEO purposes.  
No h1, h2 or h3 tag detected on this page.

0/100

#84

### ! You should define a 'description' meta tag

The page should define a unique description.

#### Description in search engines

The description of the page may be directly displayed in search engine results pages (SERP):

[Amazon.com: Online Shopping for Electronics, Apparel, Computers ...](#)

<https://www.amazon.com/> Traduire cette page

Online retailer of books, movies, music and games along with electronics, toys, apparel, sports, tools, groceries and general home and garden items. Region 1 ...

It allows you to control at best the entry preview in search engines, and to improve the click rate to your page [Learn more](#).

#### How to define a page's description?

Use `<meta name="description" content="page description">` and place it in the `<head>` tag.

No `<meta>` `description` has been found on this page. Please provide a `<meta>` `description`.

0/100

#2503

### ! robots.txt file should be defined

Indicate to web crawlers which URLs should be explored on your website.

#### The robots.txt file

Place your robots.txt file in the root of the website. It will be interpreted by the robots in charge of your SEO. It delivers instructions to specify the pages to explore by robots, like Google bot.

Note that these directives are indicative only. A lambda robot will not be blocked by the restrictions specified by the file.

We have not detected the robots.txt file on this website, you should define one:

- <https://linguplay-test.onrender.com/robots.txt>

## The other tips



0/100

#2399

### Your site doesn't use Open Graph properties

You can help social networks understand information related to the page by using Open Graph properties.

#### The Open Graph properties explained

Several properties allow social networks to learn more about the page's content. We recommend using at least the required properties:

- `<meta property="og:title" content="The title" />` [Example](#)
- `<meta property="og:type" content="The type" />` [Example](#)
- `<meta property="og:url" content="http://url.com/" />` [Example](#)
- `<meta property="og:image" content="http://image.jpg" />` [Example](#)

This information is used to improve links between your page and various social networks, including Facebook. [Read more about Open Graph here.](#)

This page does not provide information to social networks.

## Your Optimization Priorities

0/100

#2433

### The Content Security Policy is missing

Protect your website from cross-site scripting (XSS) attacks by setting up a restrictive Content-Security-Policy.

#### XSS attacks explained

XSS attacks are a type of attack in which malicious data is maliciously added to websites. The number of vulnerabilities allowing these attacks is quite large, which is why it is as useful to prevent them as to limit their harmful effects.

You can protect your pages against these attacks and their effects by restricting execution to code portions either legitimized by the domain to which they belong or by a unique integrity token. The code that does not match this security policy will not be executed and the user will be informed.

You can learn more about [XSS attacks](#) on the Open Web Application Security Project (OWASP) Website.

#### Configure a "Content-Security-Policy" (CSP) HTTP header

Set up a "Content-Security-Policy" (CSP) HTTP header to prevent or limit the damage caused by an XSS attack. To specify a security policy configure your server so the response of the first resource contains the "Content-Security-Policy" HTTP header.

Here's an example:

```
Content-Security-Policy: script-src 'self' https://apis.google.com
```

Example

In this case, only scripts coming from the current host or <https://apis.google.com> will be executed.

Read more about the CSP HTTP header by consulting [the CSP directives specification](#).

**Please, be careful, if the header is misconfigured, some of your content, scripts, or styles may be blocked. That could cause unwanted side effects. Moreover, the restrictions apply to all pages of the website.** We recommend you test the different pages of your website before deploying this header in your production environment.

No Content Security Policy on this page: it is more easily exposed to XSS attacks.

0/100

#2480

## ! This page is exposed to "clickjacking" type attacks

Keep malicious people from integrating your pages into their websites.

### Clickjacking explained

This kind of attack happens when your page gets integrated with a malicious website via <frame> or <iframe> tags. By doing this, attackers can persuade users that they are on your own page when they are not. The unsuspecting user may enter personal information that is visible on and thus vulnerable to the malicious website.

To avoid this, always indicate which domains have permission to integrate your pages.

### How to prevent clickjacking?

There are two main ways to prevent that behavior.

**1/ Configure a "X-Frame-Options" HTTP header.** Configure your server so the main resource response includes the "X-Frame-Options" HTTP header.

Three values may be defined:

- **DENY** to prevent any frame or iframe from integrating the page;
- **SAMEORIGIN** to authorize only frames from the same domain name;
- **ALLOW-FROM uri** to indicate the domains allowed to integrate a page into frame (however is not compatible with some browsers)

**2/ Define an explicit **frame-ancestors** directive into a Content-Security-Policy HTTP Header.** "frame-ancestors" directive is a newer, hence supported by fewer browsers, approach that will allow your website to authorize multiple domains instead of only the current origin. Setting this directive to 'none' is similar to **X-Frame-Options: DENY**.

Which approach to choose? If you only have the current domain to allow, do set up the two security features, for better compatibility with older browsers. If you want to allow multiple domains, you should only implement the frame-ancestors security policy.

Neither the "X-Frame-Options" HTTP header nor the "frame-ancestors" security policy are configured on this page; you are more likely to be exposed to clickjacking.

90/100

#2485

## ✓ Some third-party resources use auto detection of resource type

Protect yourself from malicious exploitation via MIME sniffing.

### MIME-Type sniffing explained

Internet Explorer and Chrome browsers have a feature called "MIME-Type sniffing" that automatically detects a web resource's type. This means, for example, that a resource identified as an image can be read as a script if its content is a script.

This property allows a malicious person to send a file to your website to inject malicious code. We advise you to disable the MIME-Type sniffing to limit such activity.

Chrome has been working on a feature called [Site Isolation](#) which provides extensive mitigation against exploitation of these types of vulnerabilities. Site Isolation is more effective when MIME types are correct.

### How to prevent MIME-Type sniffing

Configure a "X-Content-Type-Options" HTTP header. Add the "X-Content-Type-Options" HTTP header **in the responses of each resource**, associated to the "nosniff" value. It allows you to guard against such misinterpretations of your resources.

*It appears these files are hosted by a third-party, so they may not be within your control. However, you should consider any alternative to these resources to improve your page performance.*

- <https://t.contentsquare.net/uxa/532c7d0bab668.js>